

Advanced Topics in Automation and Control Engineering

Aerial Robotics Laboratory

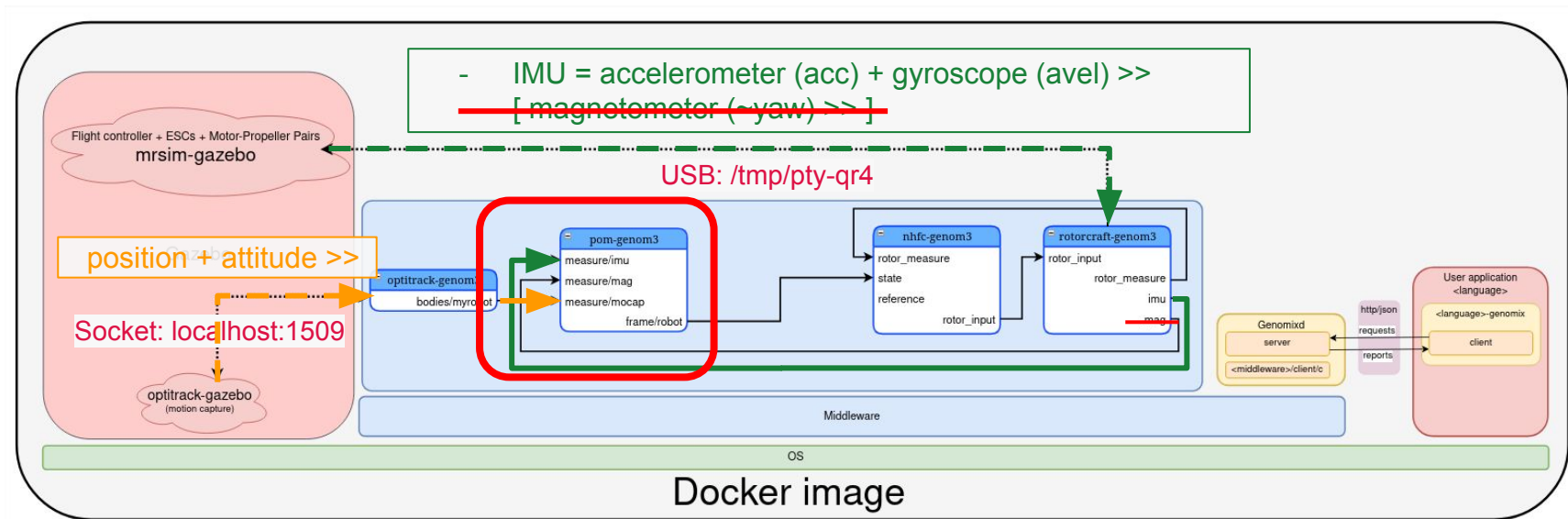
State estimation

Dr Marco Tognon
Dr Gianluca Corsini



Today...

State estimation in the Gazebo quadrotor simulation



where

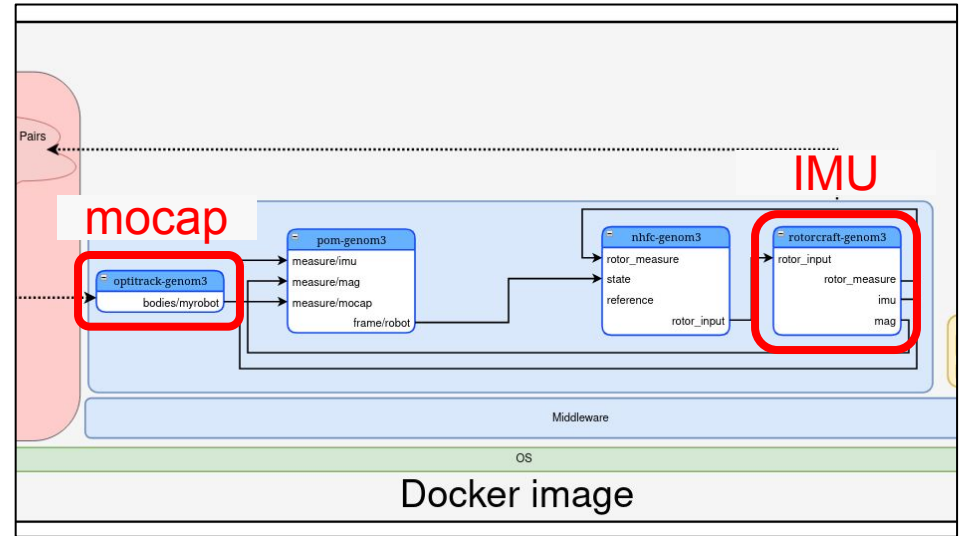
- <middleware> = [pocoLibs](#)
- <language> = python ([python-genomix documentation](#))

Let's ignore the magnetometer hereafter

Measurements – Quadrotor simulation in Gazebo

We basically have **2 sensors**:

1. **IMU** (accelerometer + gyroscope):
data are provided by `rotorcrafft-genom3`
 - a. linear acceleration (`acc`)
 - b. angular velocity (`avel`)
2. **Motion capture system**:
data are provided by `optitrack-genom3`
 - a. position (`x, y, z`)
 - b. attitude (`roll, pitch, yaw`)



State estimation – Motivation

1. Sensors have **different noise characteristics**
2. Measurements may have **different frequencies**
3. Sensors do not provide everything about the robot state
 - a. **Partial information**

Examples:

- A motion capture system is very precise, relatively high-frequency (hundreds of Hz), with minor noise (precision of 1 mm and $1e^{-1}$ deg), but provides only position and attitude
- IMU is very fast (a few KHz), noisy, and it (usually) provides only linear accelerations and angular velocities
- GPS is slow, sensitive to disturbances (noise) and it provides only position

State estimation – Goal

The goal for the state estimation is to obtain a “full” state, thus

position + attitude + linear and angular velocities + linear (and angular) accelerations

by fusing different sensor measurements.

How?

Kalman filter!

Kalman filter – Quick overview

Discrete-time state-space model

- State: $x_k = Ax_{k-1} + Bu_k + w_k$ $w_k \sim \mathcal{N}(0, Q) \rightarrow$ process noise
- Measurement: $z_k = Hx_k + v_k$ $v_k \sim \mathcal{N}(0, R) \rightarrow$ measurement noise
- Prediction step: $\hat{x}_k^- = A\hat{x}_{k-1} + Bu_k$
 $P_k^- = AP_{k-1}A^T + Q$
- Update step: $K_k = P_k^- H^T (HP_k^- H^T + R)^{-1}$
 $\hat{x}_k = \hat{x}_k^- + K_k(z_k - H\hat{x}_k^-)$
 $P_k = (I - K_k H)P_k^-$
 Q : process noise covariance
 R : measurement noise covariance

pom-genom3 – Filter model

[set_prediction_model \(attribute\)](#): Allows selecting the filter internal model. Available options are *constant-velocity* or *constant-acceleration*.

Input argument:

```
enum ::pom::prediction_model pmodel Prediction model ∈ { constant_velocity,  
constant_acceleration }
```

NB:

- The model type (constant-velocity or constant-acceleration) is set only for **translational dynamics!**
- The **rotational dynamics** is always **constant (angular) velocity**
 - Thus, angular acceleration is always zero!
 - No sensor providing angular acceleration

pom-genom3 – Add a sensor measurement z_k

1. Connect the output port of a sensor GenoM3 component to a subport of the input port **measure**

Ports

measure (multiple in)

Data structure

```
* struct ::or_pose_estimator::state measure
* struct ::or::time::ts ts
  * long sec
  * long nsec
* boolean intrinsic
* optional< struct ::or::t3d::pos > pos
  * double x
  * double y
  * double z
* optional< struct ::or::t3d::att > att
  * double qw
  * double qx
  * double qy
  * double qz
* optional< struct ::or::t3d::vel > vel
  * double vx
  * double vy
  * double vz
* optional< struct ::or::t3d::avel > avel
  * double wx
  * double wy
  * double wz
* optional< struct ::or::t3d::acc > acc
  * double ax
  * double ay
  * double az
* optional< struct ::or::t3d::aacc > aacc
  * double awx
  * double awy
  * double awz
* optional< struct ::or::t3d::pos_cov > pos_cov
  * double cov[6]
* optional< struct ::or::t3d::att_cov > att_cov
  * double cov[18]
* optional< struct ::or::t3d::att_pos_cov > att_pos_cov
  * double cov[12]
* optional< struct ::or::t3d::vel_cov > vel_cov
  * double cov[6]
* optional< struct ::or::t3d::avel_cov > avel_cov
  * double cov[6]
* optional< struct ::or::t3d::acc_cov > acc_cov
  * double cov[6]
* optional< struct ::or::t3d::aacc_cov > aacc_cov
  * double cov[6]
```

NB:

Multiple ports are ports that contain one or more subports from (to) which the component can read (write)!

pom-genom3 – Add a sensor measurement z_k

2. [add_measurement \(activity\)](#): Add a measurement source to the filter. Offset indicates the sensor position in body frame, relative to the point for which the state is estimated.

Inputs

- `string<128> port` Input port name
- `double x` (default "0") X offset (m)
- `double y` (default "0") Y offset (m)
- `double z` (default "0") Z offset (m)
- `double roll` (default "0") roll offset (rad)
- `double pitch` (default "0") pitch offset (rad)
- `double yaw` (default "0") yaw offset (rad)

pom-genom3 – Add a sensor measurement z_k

Example:

```
# read IMU from rotorcraft-genom3
>>> pom.connect_port({'local': 'measure/imu', 'remote': 'rotorcraft/imu'})
>>> pom.add_measurement('imu')

# read motion capture system from optitrack-genom3
>>> pom.connect_port({'local': 'measure/mocap', 'remote':
'optitrack/bodies/myrobot'})
>>> pom.add_measurement('mocap')
```

pom-genom3 – Obtain state estimation

- Read the output port `frame`

Example:

```
>>> result =
```

```
pom.frame('robot')['frame']
```

NB:

- When reading multiple output ports you need to specify the name of the subport you want to read!
- The multiple output port `frame` of `pom-genom3` has always one subport named `'robot'`.

Ports

`frame (multiple out)`

Data structure

```
• struct ::or_pose_estimator::state measure
  • struct ::or::time::ts ts
    • long sec
    • long nsec
  • boolean intrinsic
  • optional< struct ::or::t3d::pos > pos
    • double x
    • double y
    • double z
  • optional< struct ::or::t3d::att > att
    • double qw
    • double qx
    • double qy
    • double qz
  • optional< struct ::or::t3d::vel > vel
    • double vx
    • double vy
    • double vz
  • optional< struct ::or::t3d::avel > avel
    • double wx
    • double wy
    • double wz
  • optional< struct ::or::t3d::acc > acc
    • double ax
    • double ay
    • double az
  • optional< struct ::or::t3d::aacc > aacc
    • double awx
    • double awy
    • double awz
  • optional< struct ::or::t3d::pos_cov > pos_cov
    • double cov[6]
  • optional< struct ::or::t3d::att_cov > att_cov
    • double cov[10]
  • optional< struct ::or::t3d::att_pos_cov > att_pos_cov
    • double cov[12]
  • optional< struct ::or::t3d::vel_cov > vel_cov
    • double cov[6]
  • optional< struct ::or::t3d::avel_cov > avel_cov
    • double cov[6]
  • optional< struct ::or::t3d::acc_cov > acc_cov
    • double cov[6]
  • optional< struct ::or::t3d::aacc_cov > aacc_cov
    • double cov[6]
```

pom-genom3 – Obtain state estimation

- Parse its log file
 - We have already seen its header (state + covariances)
- For the sensor measurements, use the log file provided by its activity [log_measurements \(activity\)](#) whose content looks like as follows

```
# pom-measurements logged on Mon Apr 20 15:59:41 2026
```

```
#
```

```
# measurement #0: imu
```

```
# measurement #1: mocap
```

```
#
```

```
mid delay ts i posp attp velp avelp accp x y z qw qx qy qz vx vy vz wx wy  
wz ax ay az offx offy offz offr offp offh
```

offset of activity add_measurement

How to change covariances of the motion capture system

In `optitrack-genom3` use [`set\_noise\(attribute\)`](#)

Inputs

- `struct ::optitrack::ids::noise_s noise`
 - `double pstddev` (default `"0"`) Position white noise standard deviation
 - `double qstddev` (default `"0"`) Orientation white noise standard deviation
-

How to change covariances of the IMU

In `rotorcraft-genom3`, the covariances are stored in the IMU calibration.

To get the current calibration, use [`get\_imu\_calibration\(attribute\)`](#).

Then you can modify them.

Once modified, use [`set\_imu\_calibration\(function\)`](#) to set the new calibration in `rotorcraft-genom3`.

How to change covariances of the IMU

Example

```
>>> result = rotorcraft.get_imu_calibration()

>>> result

{'imu_calibration': {'gscale': [1, 0, 0, 0, 1, 0, 0, 0, 1], 'gbias': [0, 0,
0], 'gstddev': [0.01, 0.01, 0.01], 'ascale': [1, 0, 0, 0, 1, 0, 0, 0, 1],
'abias': [0, 0, 0], 'astddev': [0.05, 0.05, 0.05], 'mscale': [1, 0, 0, 0, 1,
0, 0, 0, 1], 'mbias': [0, 0, 0], 'mstddev': [0.05, 0.05, 0.05], 'temp': 0}}

# g: gyroscope, a: accelerometer, m: magnetometer

>>> result['imu_calibration']['astddev'] = [1, 1, 1]

>>> rotorcraft.set_imu_calibration(result)
```

Assignment

Understand how **sensor covariance tuning** affects **state estimation** by comparing different configurations against a **nominal baseline**. Focus on accuracy, smoothness, and drift behavior of the estimates.

1. Run the simulation with **default covariances** and use the plots as your reference case (**baseline**)
2. Run simulation in **“GPS + Magnetometer Mode”** and compare the results with the baseline plots
 - a. Available measurements: position (x, y, z) and yaw
 - b. Increase covariances ($\times 10$, $\times 100$)
 - i. Roll and pitch
 - ii. IMU
 - c. Keep position and yaw covariances unchanged
3. Run simulation in **“IMU-Only Mode”** and repeat the comparison
 - a. Available measurements: linear acceleration (a_x , a_y , a_z) and angular velocities (w_x , w_y , w_z)
 - b. Increase covariances ($\times 10$, $\times 100$)
 - i. Position and attitude
 - c. Keep IMU covariances unchanged

Assignment

- Use a quadrotor simulation where the robot is asked only to hover at $z=1\text{m}$ from the ground
- Generate plots as in the table below
 - Each table is a figure window, and each cell a subplot
 - State estimation data (**pom-genom3**) is plotted with continuous lines
 - Use RGB for XYZ axes, respectively
 - Set limits along plot y axes (ymin, ymax)
 - Start plots from time $t=0$
 - Variances are the diagonal elements of the covariance matrix P

pos (x,y,z)	att (roll,pitch,yaw)
vel (vx,vy,vz)	avel (wx,wy,wz)
acc (ax,ay,az)	

pos var (x,y,z)	att var (roll,pitch,yaw)
vel var (vx,vy,vz)	ang vel var (wx,wy,wz)
acc var (ax,ay,az)	